Incident Report: Sudden S3 403 Access Denied

SEV-2

Reporter: Marco (App Team)

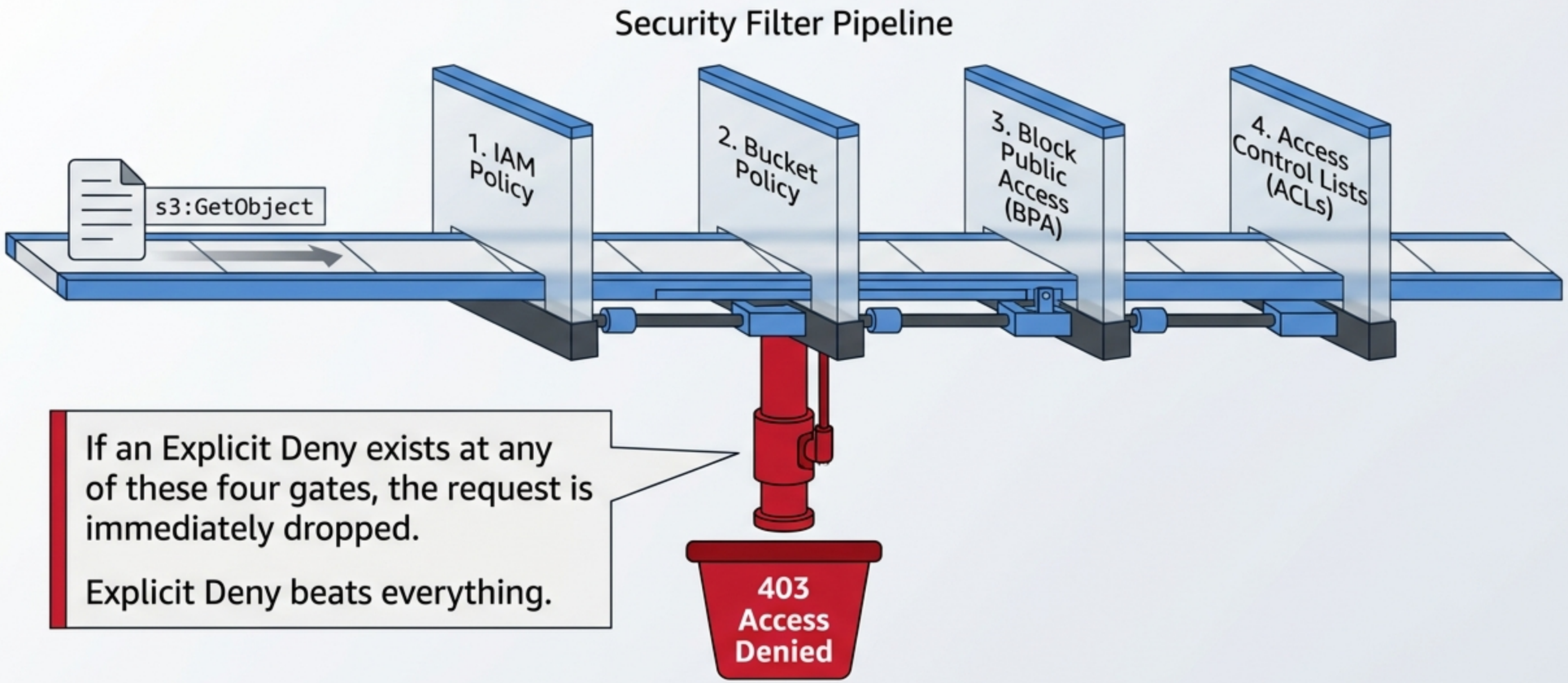
Issue Description: Reading customer statements from S3 suddenly returns 403. Worked yesterday.

Investigation Mandate

- [ ] Identify the root cause of the sudden access failure.
- [ ] Audit the S3 Access Model.
- [ ] Establish Day-2 operational standards for S3 troubleshooting.

NotebookLM

The S3 Access Pipeline Requires Passing Four Distinct Gates

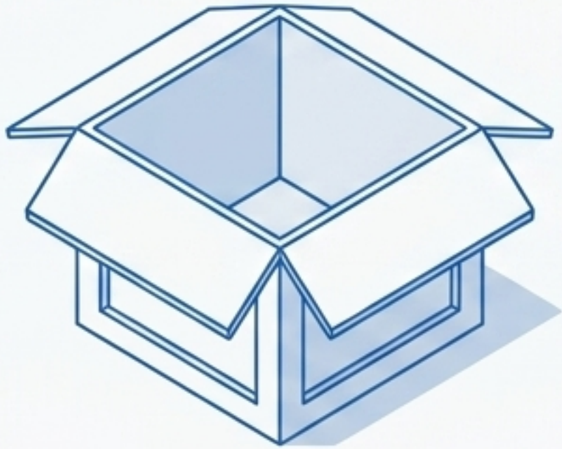


NotebookLM

The #1 Policy Typo: Misidentifying the Resource Target

(The Container)

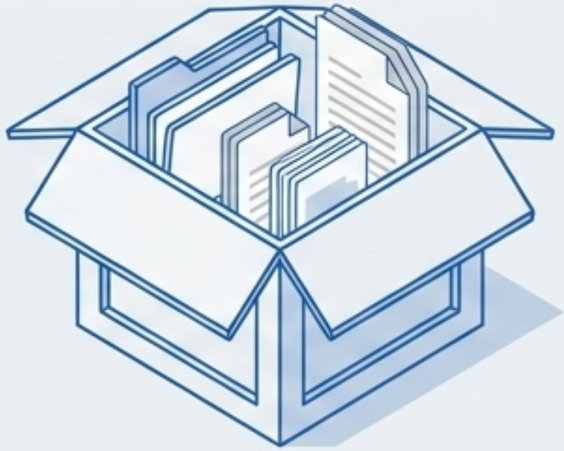
```
arn:aws:s3:::mybucket
```



Targets the bucket itself. Required for bucket-level actions like s3:ListBucket. Applying object-level rules here causes them to misfire.

(The Contents)

```
arn:aws:s3:::mybucket/*
```



Targets the objects inside the bucket. Required for object-level actions like s3:GetObject (reading customer statements).

Applying an Explicit Deny to the wrong ARN level locks the system down unexpectedly.

NotebookLM

Incident Timeline
Phase 1: Simulation & Discovery

```
$ cd labs/10-s3
$ ./break.sh
```

```
[Read attempt] -> 403
```

```
$ get-bucket-policy
```

Step 1: Environment Setup

Automates the creation of a test bucket and injects a bad Explicit Deny rule on s3:GetObject.

Step 2: The Symptom

Marco's exact issue is replicated. The app cannot read the statements.

Step 3: The Discovery

Bypasses the IAM check and pulls the bucket's internal rules, revealing the active Explicit Deny statement blocking the read requests.

NotebookLM

Incident Timeline
Phase 2: Remediation & Teardown

```
$ delete-bucket-policy
```

```
$ s3 cp
```

```
$ s3 rb --force
```

Step 4: The Surgical Fix

Removes the bad Deny statement entirely.

Crucial Note: Block Public Access (BPA) remains strictly ON during this operation.

Step 5: Verification

The copy command successfully executes and prints the real file content. Access is restored.

Step 6: Environment Cleanup

Safely deletes the entire test bucket and its contents.

NotebookLM

Day-2 Operations: S3 Troubleshooting **Golden Rules**

Hierarchy of Rules

Explicit Deny in a bucket policy beats everything.

Always walk all four S3 access layers—never assume IAM is the only gatekeeper.

Do / Don’t



Fix 403 errors surgically by locating and removing the specific bad Deny statement.



Never use “make public” as a debugging crutch to bypass a 403 error.

Keep Block Public Access ON — always.

NotebookLM